RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Department of Artificial Intelligence and Data Science

V Semester B. Tech AIDS

AD23532 - Principles of Data Science

Academic Year: 2026-2027 **Batch:** 2024-2028

Ex 6a: Exploratory Data Analysis

Register Number: 241801187 **Date:** 09-10-2026

Dataset: Netflix Titles (Kaggle) **Tools:** Python, Pandas, NumPy, Matplotlib, Seaborn

Learning Objective

To understand and perform Exploratory Data Analysis (EDA) on a real-world Netflix titles dataset using Python.

Problem Statement

Netflix maintains a large collection of movies and TV shows with details such as title, type, release year, date added, country, rating, duration, genre, director, cast and description. As a Data Scientist, the task is to analyse the content distribution, ratings, release and addition trends, durations, countries, genres, directors, actors and descriptions, using suitable statistics and visualisations, to find patterns and generate meaningful insights.

Task 1: Load the Netflix titles dataset into a DataFrame

In [1]:

```
import re, random
from collections import Counter
import numpy as np
import pandas as pd
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image, ImageDraw, ImageFont
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS

sns.set_theme(style='whitegrid')
pd.set_option('display.width', 110); pd.set_option('display.max_columns', None)
RED = '#E50914'

df = pd.read_csv('netflix_titles.csv')
print("Dataset loaded successfully")
print("Shape:", df.shape)
```

Out [1]:

```
Dataset loaded successfully
Shape: (8807, 12)
```

> **Observation:** The Netflix titles file has 8,807 titles and 12 columns: show_id, type, title, director, cast, country, date_added, release_year, rating, duration, listed_in and description.

Task 2.1: Explore the dataset - structure

In [2]:

```
print("Columns:", list(df.columns))
print()
df.info()
print()
print(df[['show_id', 'type', 'title', 'country', 'release_year', 'rating', 'duration']].head(5).to_string())
```

Out [2]:

```
Columns: ['show_id', 'type', 'title', 'director', 'cast', 'country', 'date_added', 'release_year', 'rating', 'duration', 'listed_i

<class 'pandas.DataFrame'>
RangeIndex: 8807 entries, 0 to 8806
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype
```

```
---  ------        --------------  -----
 0   show_id       8807 non-null   str
 1   type          8807 non-null   str
 2   title         8807 non-null   str
 3   director      6173 non-null   str
 4   cast          7982 non-null   str
 5   country       7976 non-null   str
 6   date_added    8797 non-null   str
 7   release_year  8807 non-null   int64
 8   rating        8803 non-null   str
 9   duration      8804 non-null   str
 10  listed_in     8807 non-null   str
 11  description   8807 non-null   str
dtypes: int64(1), str(11)
memory usage: 825.8 KB
```

	show_id	type	title	country	release_year	rating	duration
0	s1	Movie	Dick Johnson Is Dead	United States	2020	PG-13	90 min
1	s2	TV Show	Blood & Water	South Africa	2021	TV-MA	2 Seasons
2	s3	TV Show	Ganglands	NaN	2021	TV-MA	1 Season
3	s4	TV Show	Jailbirds New Orleans	NaN	2021	TV-MA	1 Season
4	s5	TV Show	Kota Factory	India	2021	TV-MA	2 Seasons

> **Observation:** Only release_year is numeric. All other columns are text, so date_added and duration must be converted before they can be analysed. Several columns (director, country, rating) already show missing values in the non-null counts.

Task 2.2: Explore the dataset - statistics, missing values and duplicates

In [3]:

```
print(df.describe().round(1).to_string())
print("\nMissing values per column:")
print(df.isnull().sum().to_string())
print("\nDuplicate rows         :", df.duplicated().sum())
print("Duplicate show_id      :", df['show_id'].duplicated().sum())
print("\nUnique values: type =", df['type'].nunique(), "| rating =", df['rating'].nunique(),
      "| release years =", df['release_year'].nunique())
```

Out [3]:

```
       release_year
count        8807.0
mean         2014.2
std             8.8
min          1925.0
25%          2013.0
50%          2017.0
75%          2019.0
max          2021.0

Missing values per column:
show_id            0
type               0
title              0
director        2634
cast             825
country          831
date_added        10
release_year       0
rating             4
duration           3
listed_in          0
description        0

Duplicate rows         : 0
Duplicate show_id      : 0

Unique values: type = 2 | rating = 17 | release years = 74
```

> **Observation:** There are no duplicate rows or duplicate show_id values. Director is missing for 2,634 titles (29.9%), country for 831, cast for 825, date_added for 10, rating for 4 and duration for 3. Release years run from 1925 to 2021, with a median of 2017, so most of the library is recent.

Task 2.3: Clean the dataset

In [4]:

```
df = df.copy()
for c in df.select_dtypes(exclude='number').columns:        # remove stray spaces
    df[c] = df[c].str.strip()

# 3 movies have their duration typed into the rating column -> move it to the duration column
shifted = df['rating'].str.contains('min', na=False)
print("Rows with duration stored in rating:", shifted.sum())
df.loc[shifted, 'duration'] = df.loc[shifted, 'rating']
df.loc[shifted, 'rating'] = np.nan

for c in ['director', 'cast', 'country', 'rating']:           # missing text -> 'Unknown'
    df[c] = df[c].fillna('Unknown')

df['date_added'] = pd.to_datetime(df['date_added'], format='%B %d, %Y')   # 10 missing stay as NaT
df['year_added'] = df['date_added'].dt.year.astype('Int64')
df['month_added'] = df['date_added'].dt.month_name()
df['duration_value'] = df['duration'].str.extract(r'(\d+)').astype(float)

print("Missing values after cleaning:")
print(df[['director', 'cast', 'country', 'rating', 'duration', 'date_added']].isnull().sum().to_string())
```

Out [4]:

```
Rows with duration stored in rating: 3
Missing values after cleaning:
director       0
cast           0
country        0
rating         0
duration       0
date_added    10
```

> **Observation:** Three Louis C.K. movies had their running time (66, 74 and 84 min) typed into the rating column and left the duration blank. These were moved to the duration column, which also explains the 3 missing durations. Missing director, cast, country and rating values were set to "Unknown", date_added was converted to a date (10 stay missing) and year, month and numeric duration columns were added.

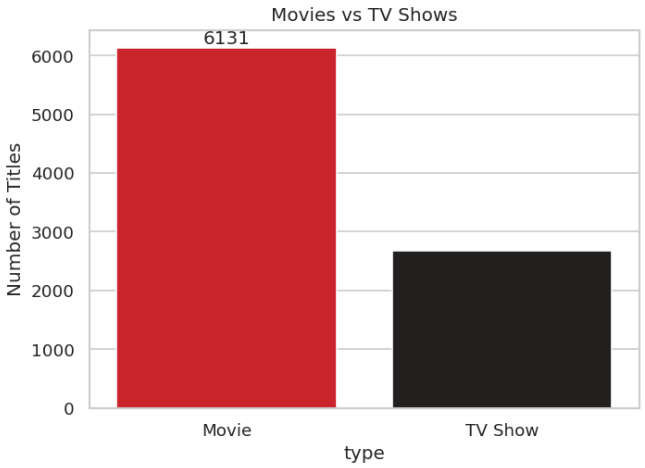
Task 3a: Movies vs TV Shows

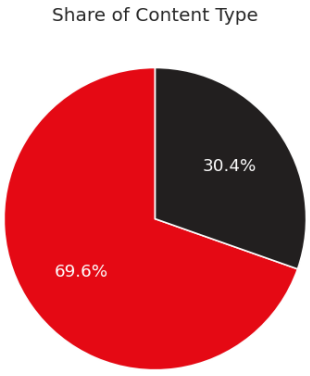
In [5]:

```
type_counts = df['type'].value_counts()
print(type_counts.to_string())
print((type_counts / len(df) * 100).round(2).to_string())
fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))
sns.barplot(x=type_counts.index, y=type_counts.values, hue=type_counts.index, palette=[RED, '#221F1F'], legend=False, ax=axes[0])
axes[0].bar_label(axes[0].containers[0]); axes[0].set_title('Movies vs TV Shows'); axes[0].set_ylabel('Number of Titles')
axes[1].pie(type_counts.values, labels=type_counts.index, autopct='%1.1f%%', colors=[RED, '#221F1F'],
            textprops={'color': 'white', 'fontsize': 11}, startangle=90)
axes[1].set_title('Share of Content Type')
plt.tight_layout(); plt.show()
```

Out [5]:

```
type
Movie      6131
TV Show    2676
type
Movie      69.62
TV Show    30.38
```





Observation: The library has 6,131 movies (69.62%) and 2,676 TV shows (30.38%), so there are about 2.3 movies for every TV show.

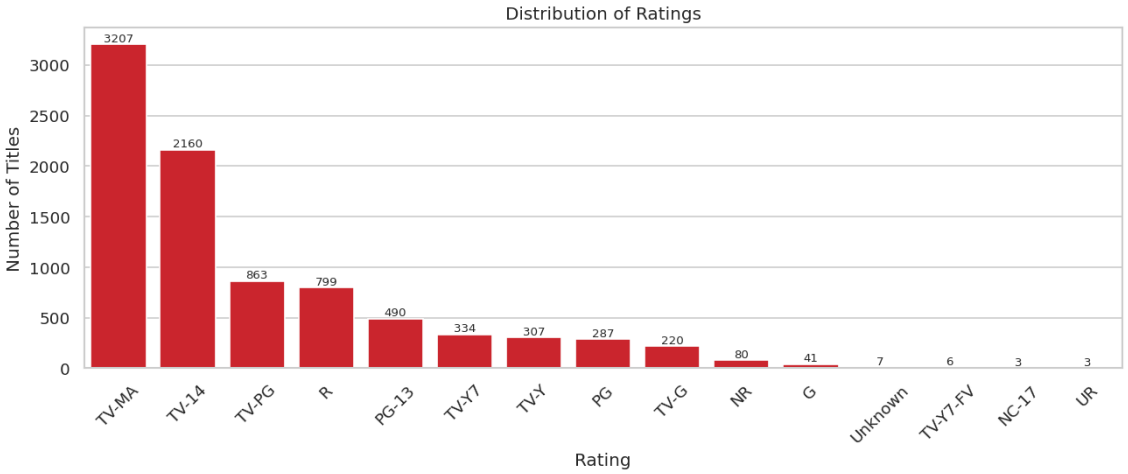
Task 3b: Distribution of ratings

In [6]:

```
rating_counts = df['rating'].value_counts()
print(rating_counts.to_string())
plt.figure(figsize=(11, 4.8))
ax = sns.barplot(x=rating_counts.index, y=rating_counts.values, color=RED)
ax.bar_label(ax.containers[0], fontsize=8)
plt.title('Distribution of Ratings'); plt.xlabel('Rating'); plt.ylabel('Number of Titles')
plt.xticks(rotation=45); plt.tight_layout(); plt.show()
```

Out [6]:

```
rating
TV-MA       3207
TV-14       2160
TV-PG        863
R            799
PG-13        490
TV-Y7        334
TV-Y         307
PG           287
TV-G         220
NR            80
G             41
Unknown        7
TV-Y7-FV       6
NC-17          3
UR             3
```



Observation: TV-MA is the most common rating (3,207 titles), followed by TV-14 (2,160), TV-PG (863), R (799) and PG-13 (490). Most content is aimed at mature or teen audiences, while the children's ratings (TV-Y7, TV-Y, TV-G, G) are much smaller. 7 titles have no rating.

Task 3c: Countries with the most Netflix content

In [7]:

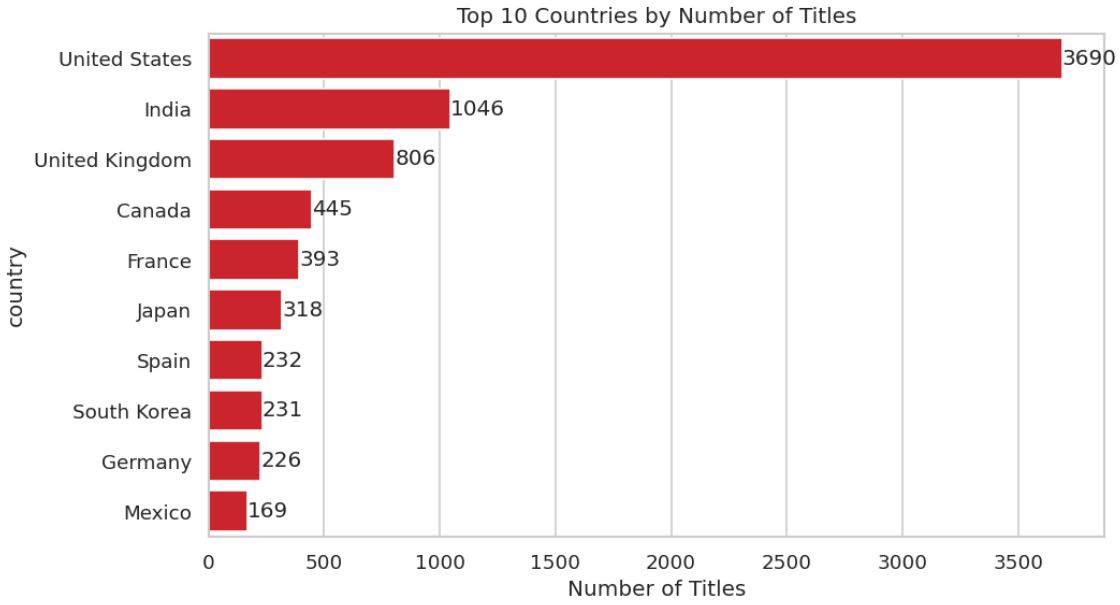
```
countries = df['country'].str.split(',').explode().str.strip()      # a title can have several countries
countries = countries[(countries != '') & (countries != 'Unknown')]
top_countries = countries.value_counts().head(10)
print(top_countries.to_string())
print("\nUnique countries:", countries.nunique())
plt.figure(figsize=(9, 5))
ax = sns.barplot(x=top_countries.values, y=top_countries.index, color=RED)
ax.bar_label(ax.containers[0]); plt.title('Top 10 Countries by Number of Titles'); plt.xlabel('Number of Titles')
plt.tight_layout(); plt.show()
```

Out [7]:

```
country
United States     3690
India             1046
United Kingdom     806
Canada             445
France             393
Japan              318
Spain              232
South Korea        231
Germany            226
```

```
Mexico            169

Unique countries: 122
```



> **Observation:** The United States has the most titles (3,690), followed by India (1,046), the United Kingdom (806), Canada (445) and France (393). The US alone has about 3.5 times as many titles as India. A title with several countries is counted once for each, and 122 countries appear in total.

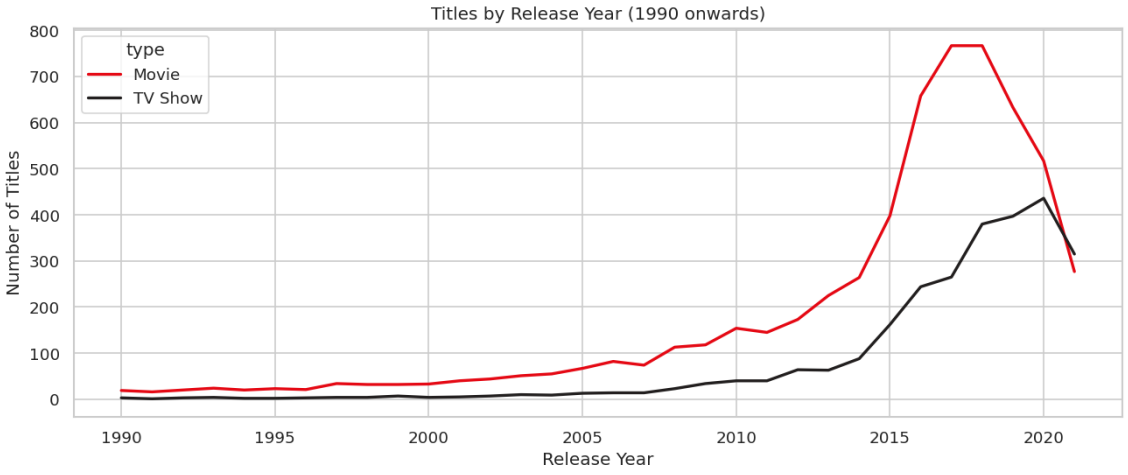
Task 4a: Years in which most titles were released

In [8]:

```
rel = df['release_year'].value_counts().sort_index()
print("Top 10 release years:"); print(df['release_year'].value_counts().head(10).to_string())
rel_type = df.groupby(['release_year', 'type']).size().unstack(fill_value=0)
plt.figure(figsize=(11, 4.8))
rel_type.loc[1990:].plot(ax=plt.gca(), color=[RED, '#221F1F'], linewidth=2)
plt.title('Titles by Release Year (1990 onwards)'); plt.xlabel('Release Year'); plt.ylabel('Number of Titles')
plt.tight_layout(); plt.show()
```

Out [8]:

```
Top 10 release years:
release_year
2018    1147
2017    1032
2019    1030
2020     953
2016     902
2021     592
2015     560
2014     352
2013     288
2012     237
```



Observation: Most titles were released in 2018 (1,147), then 2017 (1,032), 2019 (1,030), 2020 (953) and 2016 (902). Releases rise sharply after 2010 for both movies and TV shows. The 2021 count (592) is lower because the data ends in September 2021, and the dip after 2018 may also reflect titles from recent years not yet being added.

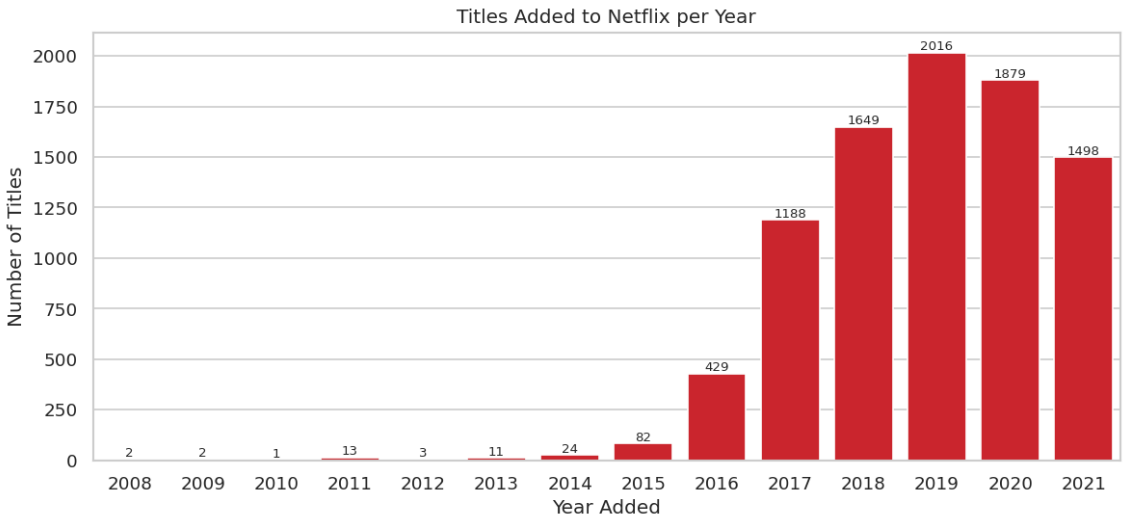
Task 4b: Years in which titles were added to Netflix

In [9]:

```
added = df['year_added'].value_counts().sort_index()
print(added.to_string())
plt.figure(figsize=(10, 4.8))
ax = sns.barplot(x=added.index.astype(int), y=added.values, color=RED)
ax.bar_label(ax.containers[0], fontsize=8)
plt.title('Titles Added to Netflix per Year'); plt.xlabel('Year Added'); plt.ylabel('Number of Titles')
plt.tight_layout(); plt.show()
```

Out [9]:

```
year_added
2008       2
2009       2
2010       1
2011      13
2012       3
2013      11
2014      24
2015      82
2016     429
2017    1188
2018    1649
2019    2016
2020    1879
2021    1498
```



Observation: Titles were added mostly from 2016 onwards. The peak was 2019 (2,016 titles), then 2020 (1,879) and 2018 (1,649). 2021 has 1,498 titles even though the data stops on 25 September. Before 2015 fewer than 25 titles were added in any year.

Task 4c: Growth of the Netflix library over time

In [10]:

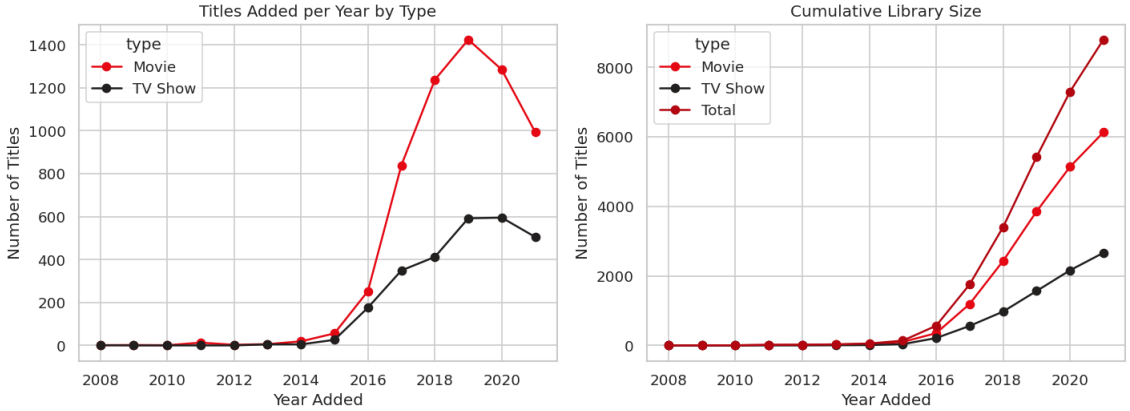
```
by_year = df.groupby(['year_added', 'type']).size().unstack(fill_value=0)
cumulative = by_year.cumsum()
cumulative['Total'] = cumulative.sum(axis=1)
print(cumulative.astype(int).to_string())
growth = cumulative['Total'].pct_change() * 100
print("\nYear-on-year growth of the library (%):"); print(growth.round(1).dropna().to_string())
fig, axes = plt.subplots(1, 2, figsize=(12, 4.6))
by_year.plot(ax=axes[0], marker='o', color=[RED, '#221F1F']); axes[0].set_title('Titles Added per Year by Type')
cumulative[['Movie', 'TV Show', 'Total']].plot(ax=axes[1], marker='o', color=[RED, '#221F1F', '#B20710'])
axes[1].set_title('Cumulative Library Size')
for a in axes: a.set_xlabel('Year Added'); a.set_ylabel('Number of Titles')
plt.tight_layout(); plt.show()
```

Out [10]:

```
type        Movie  TV Show  Total
year_added
2008            1        1      2
2009            3        1      4
```

```
2010           4        1      5
2011          17        1     18
2012          20        1     21
2013          26        6     32
2014          45       11     56
2015         101       37    138
2016         354      213    567
2017        1193      562   1755
2018        2430      974   3404
2019        3854     1566   5420
2020        5138     2161   7299
2021        6131     2666   8797

Year-on-year growth of the library (%):
year_added
2009    100.0
2010     25.0
2011    260.0
2012     16.7
2013     52.4
2014     75.0
2015    146.4
2016    310.9
2017    209.5
2018     94.0
2019     59.2
2020     34.7
2021     20.5
```



> **Observation:** The library grew from 138 titles at the end of 2015 to 8,797 by September 2021. Year-on-year growth was fastest in 2016 (310.9%) and 2017 (209.5%) and then slowed to 34.7% in 2020 and 20.5% in 2021, as the base got larger. Movie additions fell from 1,424 in 2019 to 1,284 in 2020, while TV show additions stayed level (592 and 595).

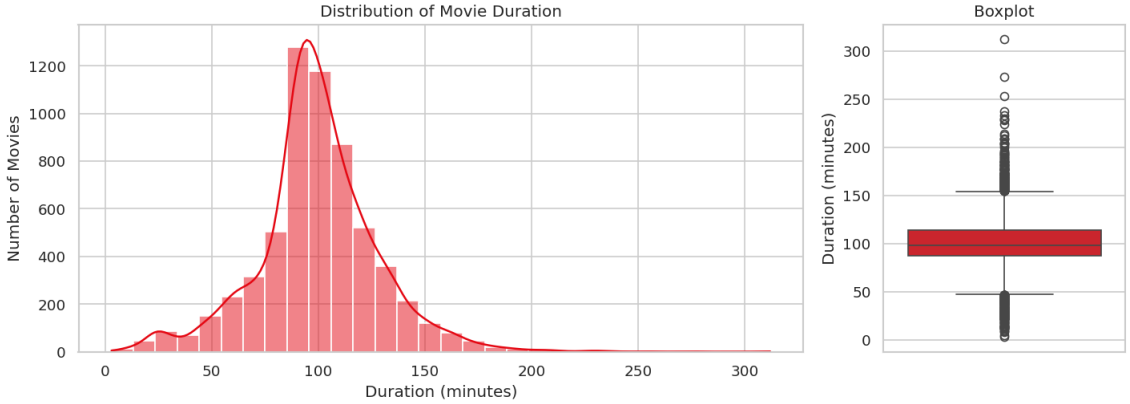
Task 5a: Distribution of movie durations

In [11]:

```
movies = df[df['type'] == 'Movie']
dur = movies['duration_value']
print(dur.describe().round(1).to_string())
print("Skewness:", round(dur.skew(), 2))
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5), gridspec_kw={'width_ratios': [3, 1]})
sns.histplot(dur, bins=30, kde=True, color=RED, ax=axes[0]); axes[0].set_title('Distribution of Movie Duration')
axes[0].set_xlabel('Duration (minutes)'); axes[0].set_ylabel('Number of Movies')
sns.boxplot(y=dur, color=RED, ax=axes[1]); axes[1].set_title('Boxplot'); axes[1].set_ylabel('Duration (minutes)')
plt.tight_layout(); plt.show()
```

Out [11]:

```
count    6131.0
mean       99.6
std        28.3
min         3.0
25%        87.0
50%        98.0
75%       114.0
max       312.0
Skewness: 0.2
```



Observation: Movie durations are close to a normal curve: the mean is 99.6 minutes, the median is 98 and the skewness is 0.2. Half of the movies last between 87 and 114 minutes. The boxplot shows outliers on both sides, from very short films (minimum 3 minutes) to very long ones (maximum 312 minutes).

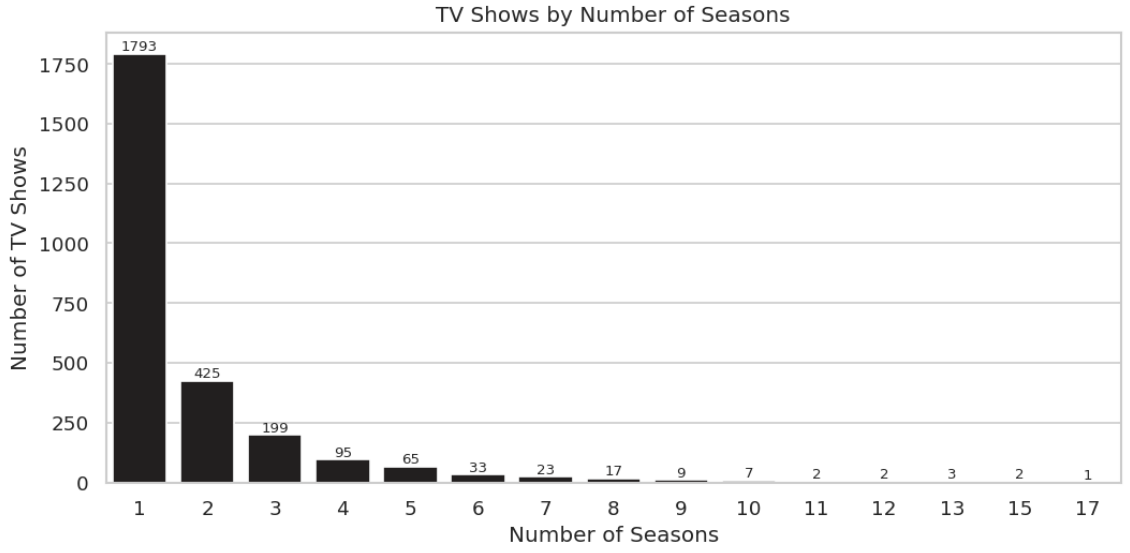
Task 5b: TV shows per number of seasons

In [12]:

```
seasons = df[df['type'] == 'TV Show']['duration_value'].astype(int).value_counts().sort_index()
print(seasons.to_string())
print("Share of single-season shows: %.1f%%" % (seasons.loc[1] / seasons.sum() * 100))
plt.figure(figsize=(9, 4.6))
ax = sns.barplot(x=seasons.index, y=seasons.values, color='#221F1F')
ax.bar_label(ax.containers[0], fontsize=8)
plt.title('TV Shows by Number of Seasons'); plt.xlabel('Number of Seasons'); plt.ylabel('Number of TV Shows')
plt.tight_layout(); plt.show()
```

Out [12]:

```
duration_value
1     1793
2      425
3      199
4       95
5       65
6       33
7       23
8       17
9        9
10       7
11       2
12       2
13       3
15       2
17       1
Share of single-season shows: 67.0%
```



Observation: Most TV shows have only one season: 1,793 of 2,676 (67.0%). The number falls quickly with more seasons (425 with two, 199 with three), and very few shows have more than 10 seasons (the maximum is 17).

Task 6a: Top 10 genres

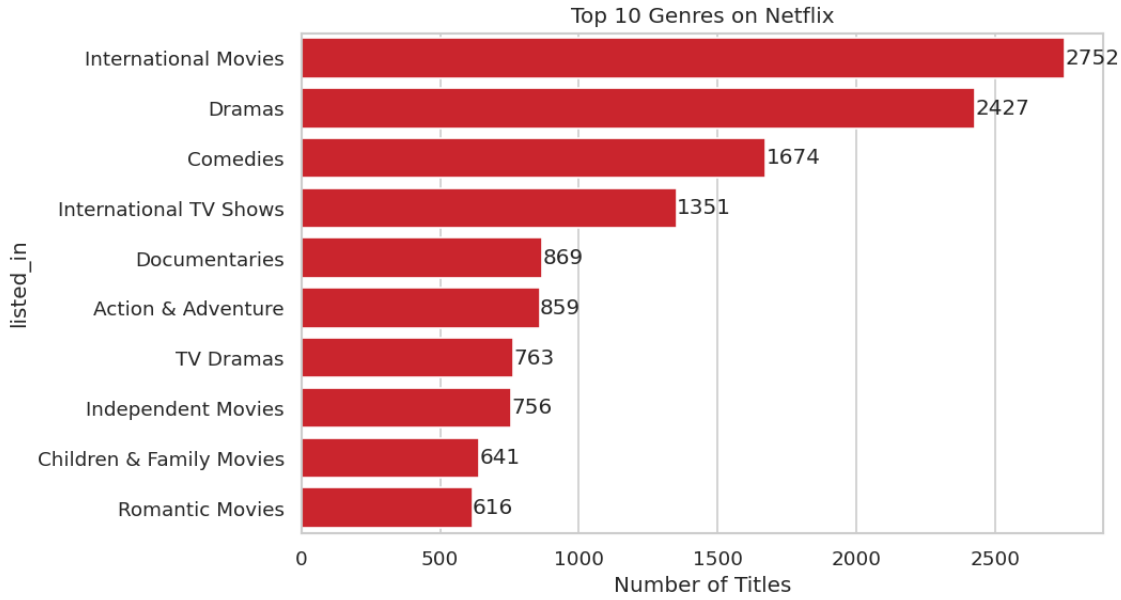
In [13]:

```
genres = df['listed_in'].str.split(',').explode().str.strip()
top_genres = genres.value_counts().head(10)
print(top_genres.to_string())
print("\nUnique genres:", genres.nunique())
plt.figure(figsize=(9, 5))
ax = sns.barplot(x=top_genres.values, y=top_genres.index, color=RED)
ax.bar_label(ax.containers[0]); plt.title('Top 10 Genres on Netflix'); plt.xlabel('Number of Titles')
plt.tight_layout(); plt.show()
```

Out [13]:

```
listed_in
International Movies        2752
Dramas                      2427
Comedies                    1674
International TV Shows      1351
Documentaries                869
Action & Adventure           859
TV Dramas                    763
Independent Movies           756
Children & Family Movies     641
Romantic Movies              616

Unique genres: 42
```



> **Observation:** The top genres are International Movies (2,752), Dramas (2,427), Comedies (1,674), International TV Shows (1,351) and Documentaries (869). There are 42 distinct genres. A title can list several genres, so these counts overlap and add up to more than the number of titles.

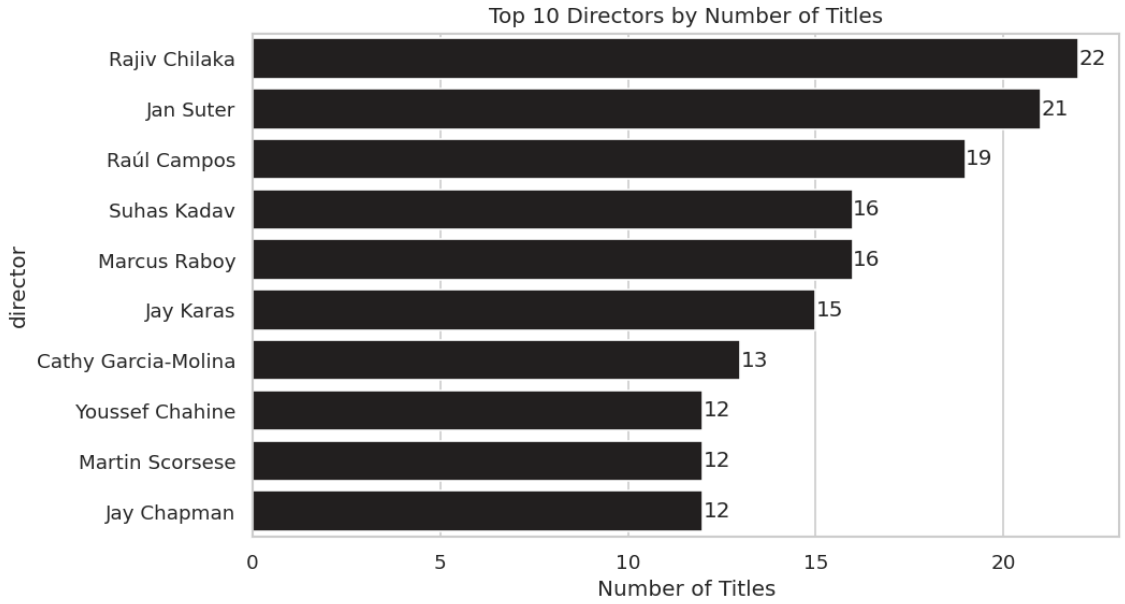
Task 6b: Directors with the most titles

In [14]:

```
directors = df['director'].str.split(',').explode().str.strip()
directors = directors[directors != 'Unknown']
top_directors = directors.value_counts().head(10)
print(top_directors.to_string())
print("\nTitles with no director listed: %d (%.1f%%)" % ((df['director'] == 'Unknown').sum(), (df['director'] == 'Unknown').mean(
plt.figure(figsize=(9, 5))
ax = sns.barplot(x=top_directors.values, y=top_directors.index, color='#221F1F')
ax.bar_label(ax.containers[0]); plt.title('Top 10 Directors by Number of Titles'); plt.xlabel('Number of Titles')
plt.tight_layout(); plt.show()
```

Out [14]:

```
director
Rajiv Chilaka          22
Jan Suter              21
Raúl Campos            19
Suhas Kadav            16
Marcus Raboy           16
Jay Karas              15
Cathy Garcia-Molina    13
Youssef Chahine        12
Martin Scorsese        12
Jay Chapman            12
Titles with no director listed: 2634 (29.9%)
```



> **Observation:** Rajiv Chilaka (22 titles), Jan Suter (21) and Raúl Campos (19) have the most titles. The counts are small, so the library is spread across many directors. 2,634 titles (29.9%) have no director listed, so this ranking is based on the other 70%.

Task 6c: Actors who appear most frequently

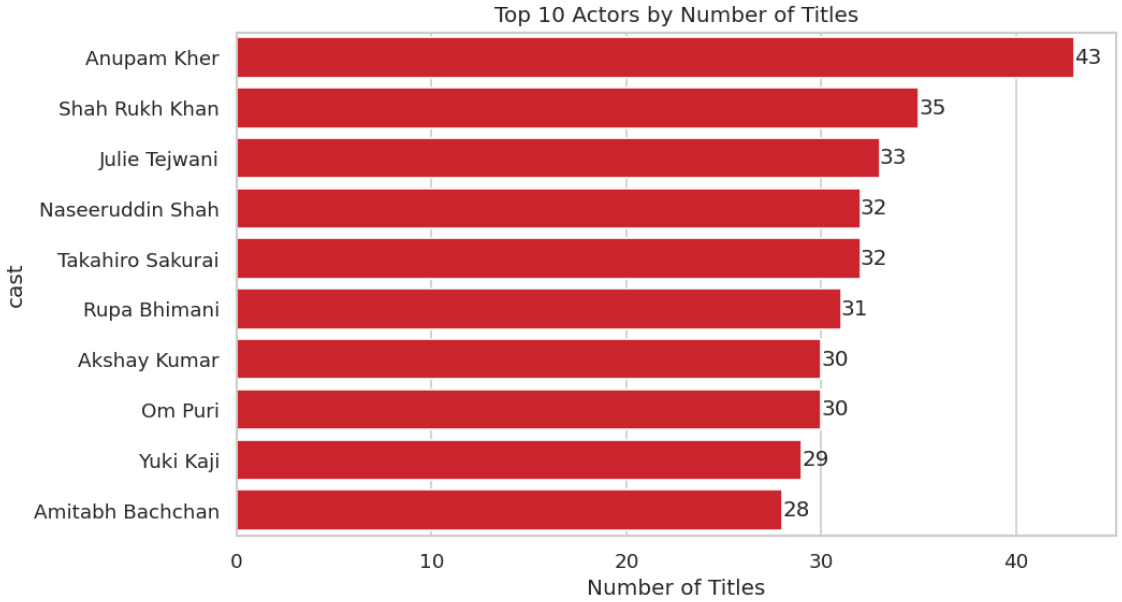
In [15]:

```
actors = df['cast'].str.split(',').explode().str.strip()
actors = actors[actors != 'Unknown']
top_actors = actors.value_counts().head(10)
print(top_actors.to_string())
print("\nUnique actors:", actors.nunique())
plt.figure(figsize=(9, 5))
ax = sns.barplot(x=top_actors.values, y=top_actors.index, color=RED)
ax.bar_label(ax.containers[0]); plt.title('Top 10 Actors by Number of Titles'); plt.xlabel('Number of Titles')
plt.tight_layout(); plt.show()
```

Out [15]:

```
cast
Anupam Kher          43
Shah Rukh Khan       35
Julie Tejwani        33
Naseeruddin Shah     32
Takahiro Sakurai     32
Rupa Bhimani         31
Akshay Kumar         30
Om Puri              30
Yuki Kaji            29
Amitabh Bachchan     28

Unique actors: 36439
```



> **Observation:** Anupam Kher (43 titles), Shah Rukh Khan (35), Julie Tejwani (33), Naseeruddin Shah (32) and Takahiro Sakurai (32) appear most often. Many of the top names are Indian actors, and several are voice actors who appear in many titles. The cast column has 36,439 unique actors, and 825 titles have no cast listed.

Task 7a: Most common words in the description field (word cloud)

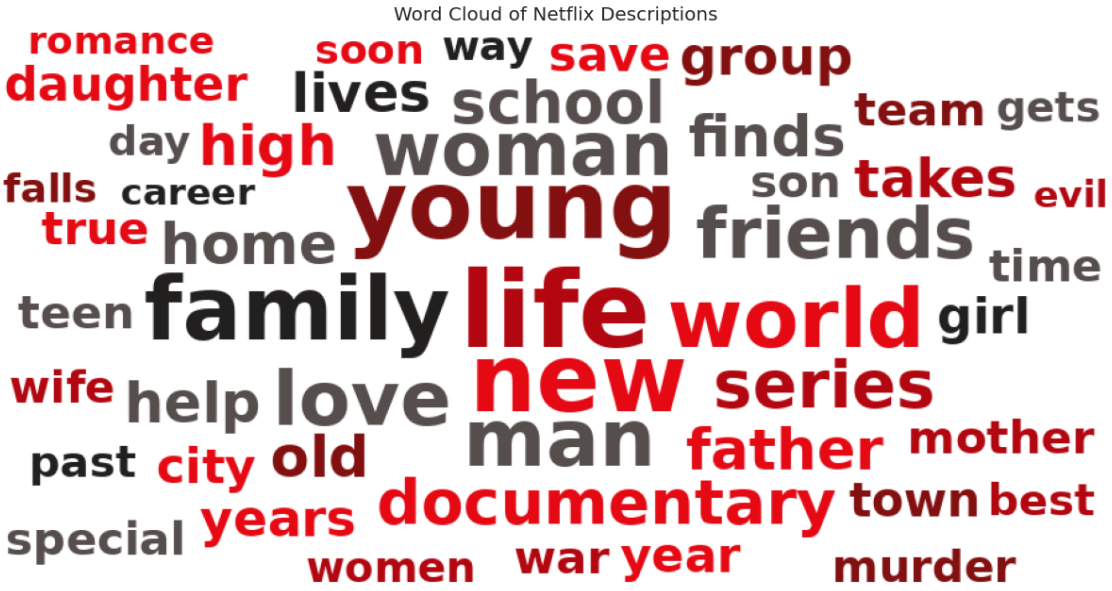
In [16]:

```
def make_wordcloud(freq, width=900, height=450, seed=1):
    """Simple word cloud: bigger words for higher frequency, placed on a spiral without overlapping."""
    rng = random.Random(seed)
    img = Image.new('RGB', (width, height), 'white'); draw = ImageDraw.Draw(img)
    used = np.zeros((height, width), dtype=bool)
    font_path = matplotlib.get_data_path() + '/fonts/ttf/DejaVuSans-Bold.ttf'
    top = max(freq.values())
    colours = ['#E50914', '#B20710', '#221F1F', '#564D4D', '#831010']
    for word, n in sorted(freq.items(), key=lambda kv: -kv[1]):
        font = ImageFont.truetype(font_path, int(14 + 72 * (n / top) ** 0.8))
        l, t, r, b = font.getbbox(word); w, h = r - l + 6, b - t + 6
        for step in range(4000):                                    # spiral outwards from the centre
            ang = 0.3 * step; rad = 0.25 * step
            x = int(width / 2 + 1.7 * rad * np.cos(ang) - w / 2); y = int(height / 2 + rad * np.sin(ang) - h / 2)
            if x < 0 or y < 0 or x + w > width or y + h > height: continue
            if not used[y:y + h, x:x + w].any():
                draw.text((x + 3 - l, y + 3 - t), word, font=font, fill=rng.choice(colours))
                used[y:y + h, x:x + w] = True; break
    return img

words = re.findall(r"[a-z]{3,}", " ".join(df['description']).lower())
words = [w for w in words if w not in ENGLISH_STOP_WORDS]
word_freq = Counter(words)
print("Total words:", len(words), "| Unique words:", len(word_freq))
print("Top 15 words:", ", ".join(f"{w} ({n})" for w, n in word_freq.most_common(15)))
cloud = make_wordcloud(dict(word_freq.most_common(80)))
plt.figure(figsize=(11, 5.5)); plt.imshow(cloud); plt.axis('off')
plt.title('Word Cloud of Netflix Descriptions'); plt.tight_layout(); plt.show()
```

Out [16]:

```
Total words: 120251 | Unique words: 18499
Top 15 words: life (845), young (729), new (709), family (635), world (570), man (538), love (504), woman (486), friends (445), se
```



Observation: After removing stopwords the descriptions contain 120,251 words (18,499 unique). The most frequent are life (845), young (729), new (709), family (635), world (570), man (538) and love (504), so the typical story is about family, love, and young people. "Series" and "documentary" also appear often. The cloud was drawn with a short function written for this record (the wordcloud package is not needed).

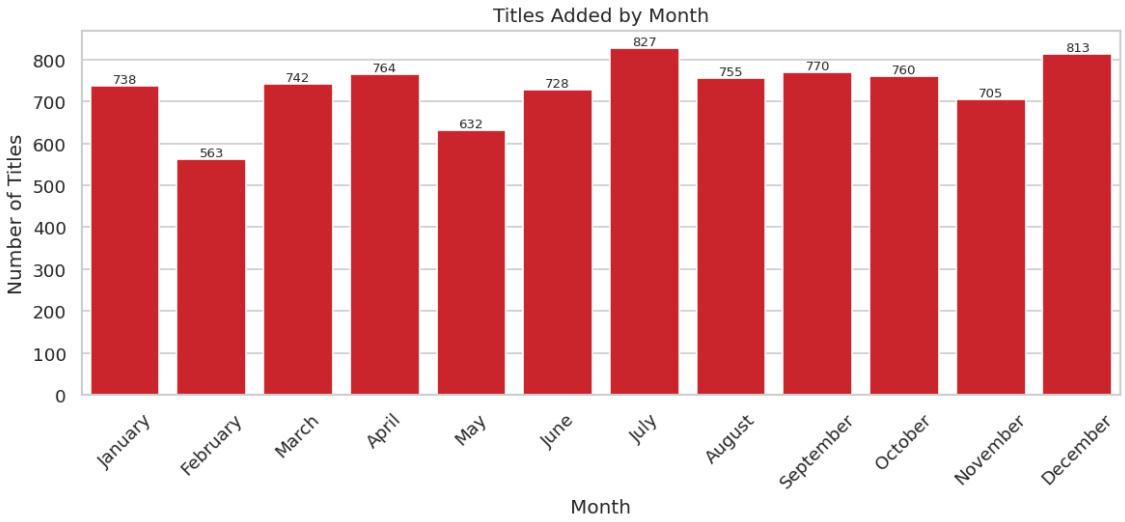
Task 8: Months in which Netflix adds the most content

In [17]:

```
months = ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August',
          'September', 'October', 'November', 'December']
month_counts = df['month_added'].value_counts().reindex(months)
print(month_counts.astype(int).to_string())
plt.figure(figsize=(10, 4.8))
ax = sns.barplot(x=month_counts.index, y=month_counts.values, color=RED)
ax.bar_label(ax.containers[0], fontsize=8)
plt.title('Titles Added by Month'); plt.xlabel('Month'); plt.ylabel('Number of Titles')
plt.xticks(rotation=45); plt.tight_layout(); plt.show()
```

Out [17]:

```
month_added
January      738
February     563
March        742
April        764
May          632
June         728
July         827
August       755
September    770
October      760
November     705
December     813
```



Observation: July has the most additions (827), followed by December (813), September (770) and April (764). February has the fewest (563). The monthly counts stay between about 560 and 830, so there is no strong seasonal pattern in when titles are added.

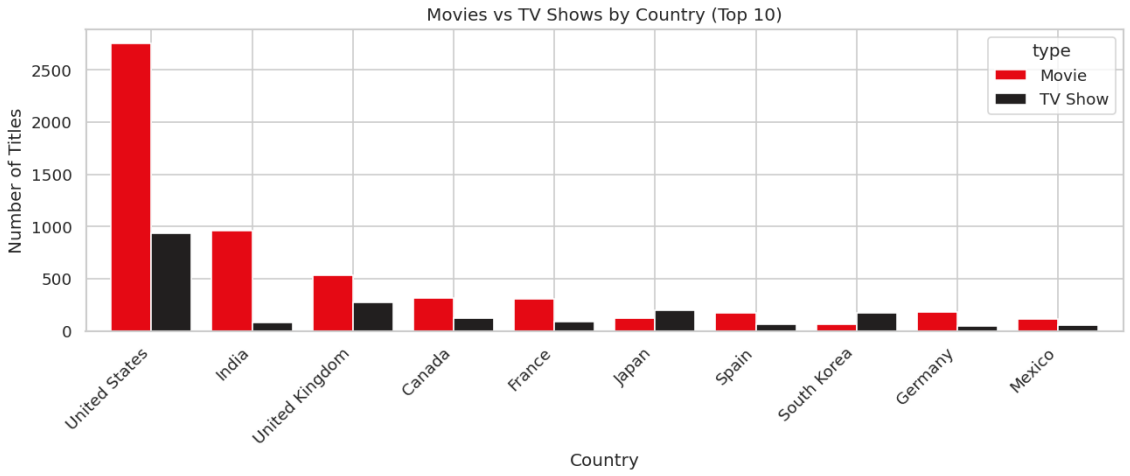
Task 9: Countries producing more Movies vs TV Shows

In [18]:

```
ct = df.assign(country=df['country'].str.split(',')).explode('country')
ct['country'] = ct['country'].str.strip()
ct = ct[(ct['country'] != '') & (ct['country'] != 'Unknown')].reset_index(drop=True)
top10 = ct['country'].value_counts().head(10).index
cmp_ = pd.crosstab(ct['country'], ct['type']).loc[top10]
cmp_['TV Show share (%)'] = (cmp_['TV Show'] / (cmp_['Movie'] + cmp_['TV Show']) * 100).round(1)
print(cmp_.to_string())
cmp_[['Movie', 'TV Show']].plot(kind='bar', figsize=(11, 4.8), color=[RED, '#221F1F'], width=0.8)
plt.title('Movies vs TV Shows by Country (Top 10)'); plt.xlabel('Country'); plt.ylabel('Number of Titles')
plt.xticks(rotation=45, ha='right'); plt.tight_layout(); plt.show()
```

Out [18]:

```
type            Movie  TV Show  TV Show share (%)
country
United States    2752      938               25.4
India             962       84                8.0
United Kingdom    534      272               33.7
Canada            319      126               28.3
France            303       90               22.9
Japan             119      199               62.6
Spain             171       61               26.3
South Korea        61      170               73.6
Germany           182       44               19.5
Mexico            111       58               34.3
```



Observation: In most of the top 10 countries movies outnumber TV shows, for example the United States (2,752 movies, 938 TV shows) and India (962 vs 84; 92% movies). Japan (62.6% TV shows) and South Korea (73.6%) are the exceptions: they produce more TV shows than movies.

Task 10: Most common ratings for Movies vs TV Shows

In [19]:

```
rt = pd.crosstab(df['rating'], df['type'])
rt = rt.loc[rt.sum(axis=1).sort_values(ascending=False).index]
print(rt.to_string())
for t in ['Movie', 'TV Show']:
    top3 = (rt[t] / rt[t].sum() * 100).sort_values(ascending=False).head(3).round(1)
    print(f"\nTop 3 ratings for {t}s:", ", ".join(f"{k} ({v}%)" for k, v in top3.items()))
rt.plot(kind='bar', figsize=(11, 4.8), color=[RED, '#221F1F'], width=0.8)
plt.title('Ratings of Movies vs TV Shows'); plt.xlabel('Rating'); plt.ylabel('Number of Titles')
plt.xticks(rotation=45); plt.tight_layout(); plt.show()
```

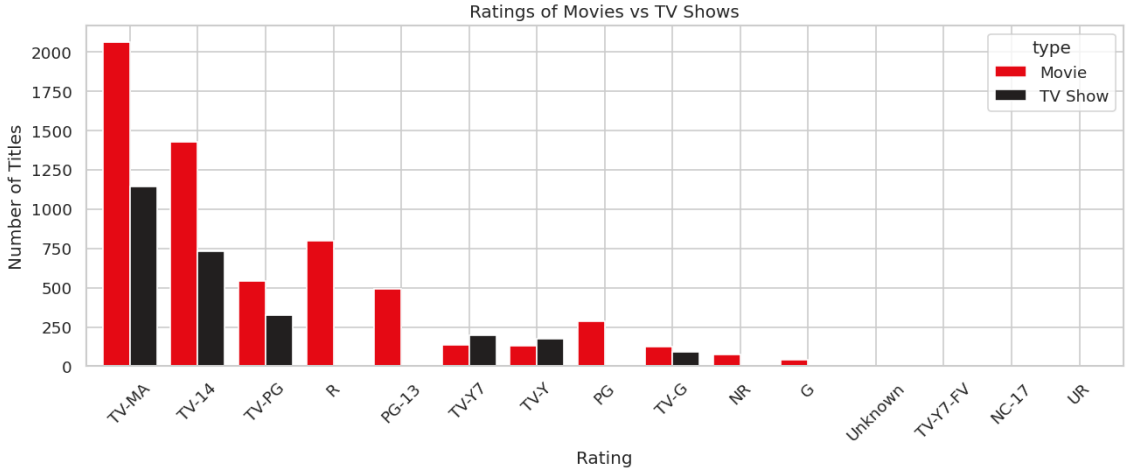
Out [19]:

```
type      Movie  TV Show
rating
TV-MA      2062     1145
TV-14      1427      733
TV-PG       540      323
R           797        2
PG-13       490        0
TV-Y7       139      195
TV-Y        131      176
```

```
PG          287        0
TV-G        126       94
NR           75        5
G            41        0
Unknown       5        2
TV-Y7-FV      5        1
NC-17         3        0
UR            3        0

Top 3 ratings for Movies: TV-MA (33.6%), TV-14 (23.3%), R (13.0%)

Top 3 ratings for TV Shows: TV-MA (42.8%), TV-14 (27.4%), TV-PG (12.1%)
```



Observation: TV-MA is the top rating for both: 33.6% of movies and 42.8% of TV shows, followed by TV-14 (23.3% and 27.4%). Movies then have R (13.0%), while TV shows have TV-PG (12.1%). The ratings R, PG-13, PG, G, NC-17 and UR are used almost only for movies, and the children's ratings TV-Y7 and TV-Y are more common for TV shows (195 and 176) than for movies (139 and 131).

Task 11: Oldest and newest titles

In [20]:

```
cols = ['title', 'type', 'release_year', 'date_added']
print("Oldest titles by release year:")
print(df.sort_values('release_year').head(5)[cols].to_string(index=False))
print("\nNewest titles by release year (", df['release_year'].max(), ") - count:", (df['release_year'] == df['release_year'].max(
print("\nOldest titles by date added to Netflix:")
print(df.sort_values('date_added').head(5)[cols].to_string(index=False))
print("\nNewest titles by date added to Netflix:")
print(df.sort_values('date_added', ascending=False).head(5)[cols].to_string(index=False))
print("\nRelease years range from", df['release_year'].min(), "to", df['release_year'].max())
```

Out [20]:

```
Oldest titles by release year:
                          title    type  release_year date_added
 Pioneers: First Women Filmmakers* TV Show          1925 2018-12-30
                   Prelude to War   Movie          1942 2017-03-31
               The Battle of Midway   Movie          1942 2017-03-31
   WWII: Report from the Aleutians   Movie          1943 2017-03-31
Why We Fight: The Battle of Russia   Movie          1943 2017-03-31

Newest titles by release year ( 2021 ) - count: 592

Oldest titles by date added to Netflix:
                     title    type  release_year date_added
       To and From New York   Movie          2006 2008-01-01
            Dinner for Five TV Show          2007 2008-02-04
    Just Another Love Story   Movie          2007 2009-05-05
                  Splatter   Movie          2009 2009-11-18
Mad Ron's Prevues from Hell   Movie          1987 2010-11-01

Newest titles by date added to Netflix:
                           title    type  release_year date_added
              Dick Johnson Is Dead   Movie          2020 2021-09-25
   My Little Pony: A New Generation   Movie          2021 2021-09-24
Vendetta: Truth, Lies and The Mafia TV Show          2021 2021-09-24
                     The Starling   Movie          2021 2021-09-24
      The Great British Baking Show TV Show          2021 2021-09-24

Release years range from 1925 to 2021
```

> **Observation:** The oldest title by release year is the TV show "Pioneers: First Women Filmmakers*" (1925), followed by WWII documentaries from 1942 and 1943 such as "Prelude to War". The newest release year is 2021 (592 titles). The first title added to Netflix was "To and From New York" (1 January 2008) and the latest was "Dick Johnson Is Dead" on 25 September 2021, so the data covers up to that date.

Task 12: Genre co-occurrence (genre correlation heatmap)

In [21]:

```
g = df['listed_in'].str.get_dummies(sep=', ')               # one 0/1 column per genre
top15 = g.sum().sort_values(ascending=False).head(15).index
corr = g[top15].corr()
plt.figure(figsize=(11, 8.5))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', vmin=-1, vmax=1, center=0, linewidths=0.4, annot_kws={'size': 7})
plt.title('Genre Correlation Heatmap (15 most common genres)')
plt.xticks(rotation=45, ha='right'); plt.tight_layout(); plt.show()

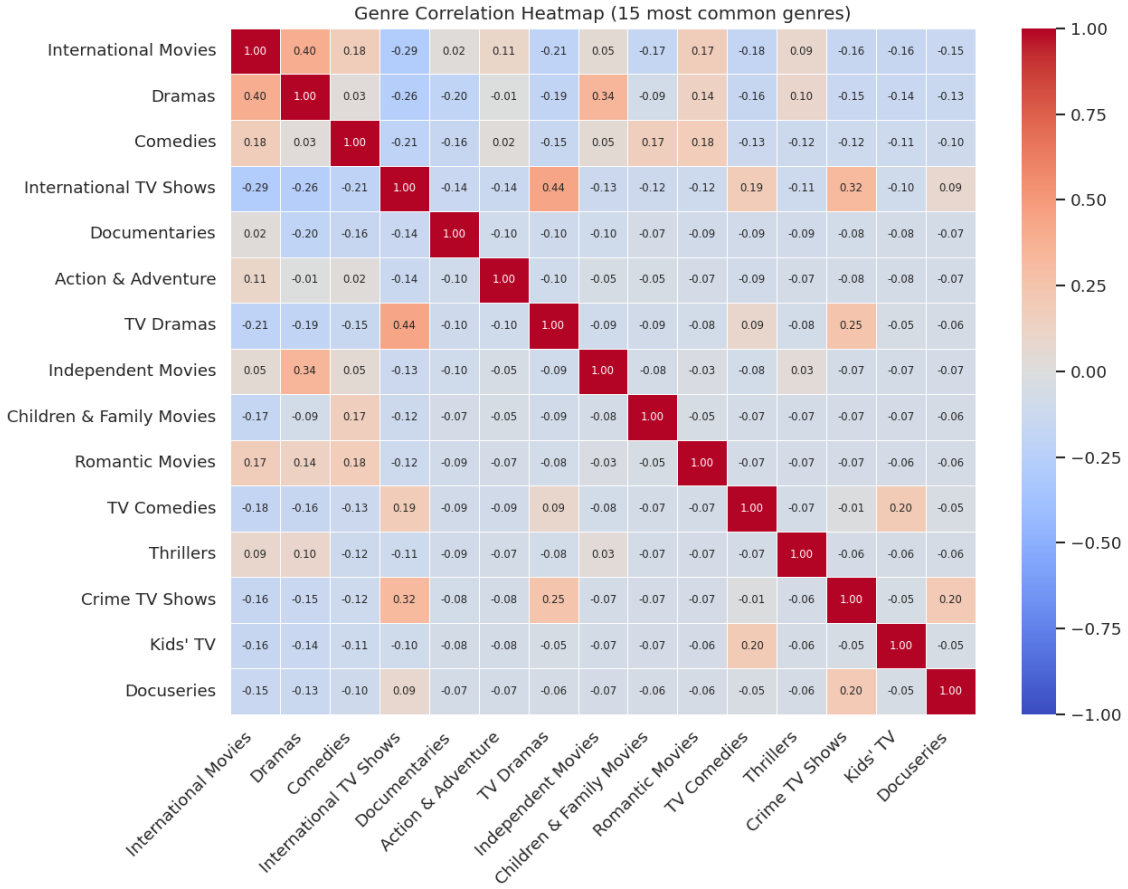
co = g.T.dot(g)                                              # how many titles share two genres
pairs = [(a, b, co.loc[a, b]) for i, a in enumerate(co.columns) for b in co.columns[i + 1:]]
pairs = sorted(pairs, key=lambda p: -p[2])[:8]
print("Most frequent genre pairs:")
for a, b, n in pairs: print(f"  {a} + {b}: {n} titles")
c = corr.where(np.triu(np.ones(corr.shape, dtype=bool), 1)).stack().dropna().sort_values()
print("\nStrongest positive correlations:"); print(c.tail(3).round(2).to_string())
print("\nStrongest negative correlations:"); print(c.head(3).round(2).to_string())
```

Out [21]:

```
Most frequent genre pairs:
  Dramas + International Movies: 1483 titles
  Comedies + International Movies: 804 titles
  Dramas + Independent Movies: 588 titles
  International TV Shows + TV Dramas: 514 titles
  Comedies + Dramas: 502 titles
  Action & Adventure + International Movies: 398 titles
  International Movies + Romantic Movies: 372 titles
  International TV Shows + Romantic TV Shows: 315 titles

Strongest positive correlations:
Dramas                 Independent Movies    0.34
International Movies   Dramas               0.40
International TV Shows  TV Dramas            0.44

Strongest negative correlations:
International Movies  International TV Shows   -0.29
Dramas                International TV Shows   -0.26
International Movies  TV Dramas                -0.21
```



Observation: The heatmap shows correlation between genre tags for the 15 most common genres. The most frequent pairs are Dramas + International Movies (1,483 titles), Comedies + International Movies (804) and Dramas + Independent Movies (588). The strongest positive correlations are International TV Shows with TV Dramas (0.44), International Movies with Dramas (0.40) and Dramas with Independent Movies (0.34). The strongest negative is International Movies with International TV Shows (-0.29): movie genres and TV genres are almost never combined.

Result

Thus an Exploratory Data Analysis was performed on the Netflix titles dataset (8,807 titles, 12 columns). After fixing three rows with a misplaced duration and filling missing text values, the analysis showed that 69.6% of the content is movies, TV-MA is the most common rating, and the United States (3,690 titles) and India (1,046) lead in content. Additions peaked in 2019 and the library grew from 138 titles in 2015 to 8,797 in September 2021, with growth slowing as the library got larger. Movies average about 100 minutes, 67% of TV shows have a single season, International Movies, Dramas and Comedies are the top genres, and drama genres co-occur most often. Japan and South Korea produce more TV shows than movies, while most other countries produce more movies.